



CLEVR Dynamic Data Grid - Complete Code Flow

Here's the comprehensive flow of how your modular CLEVR Dynamic Data Grid works from initialization to rendering:

1. Component Initialization Flow

DynamicGrid.tsx (Main Component)

```
User loads page with widget
↓
DynamicGrid component mounts
↓
useMemo triggers with dependencies:
  - dataSourceCell, dataSourceRow, dataSourceColumn
  - referenceRow, referenceColumn
  - cellAttribute, headerAttribute, rowAttribute
  - showCellAs, showRowAs, showHeaderAs
  - enableRowAggregation, enableColumnAggregation
  - aggregation functions and labels
↓
Calls buildGridData(props)
```

2. Data Processing Flow

GridBuilder.ts → DataProcessor.ts

```
buildGridData() called
↓
Step 1: DataProcessor.createMatrix(props)
├─ Gets cellItems, rowItems, columnItems from Mendix datasources
├─ Creates empty cellMatrix Map<rowKey, Map<columnKey, cellItem>>
├─ Creates empty uniqueRows and uniqueColumns Sets
├─ For each cellItem:
│   └─ Try to get associatedRow using referenceRow.get(cellItem)
│   └─ Extract entityId using ValueStatus.Available check
│   └─ Find matching row in rowItems array
│   └─ Same process for associatedColumn
│   └─ Fallback to index-based matching if associations fail
│   └─ Add to cellMatrix[rowKey][columnKey] = cellItem
├─ Convert Sets to sorted arrays
└─ Return { cellMatrix, sortedRows, sortedColumns, columnKeys }
```

▮ 3. Column Building Flow

GridBuilder.ts Column Creation

```
Step 2: buildColumns(props, matrixData)
↓
Initialize columnHelper and empty columns array
↓
IF showRowAs !== "none":
  └─ Create row header column with createRowHeaderColumn()
  └─ Cell renderer checks row.original._isAggregation
  └─ If aggregation row: show rowAggregationLabel with CSS classes
  └─ If regular row: use ContentRenderer.renderContent()
  └─ Add to columns array
↓
FOR each columnItem in sortedColumns:
  └─ Create data column with createDataColumn()
  └─ Get column header using ContentRenderer
  └─ Cell renderer logic:
    └─ IF aggregation row: calculateColumnAggregation()
    └─ ELSE: get cellItem from matrix[rowKey][columnKey]
    └─ Render cell content using ContentRenderer
    └─ Apply appropriate CSS classes
  └─ Add to columns array
↓
IF enableColumnAggregation:
  └─ Create aggregation column with createColumnAggregationColumn()
  └─ Header shows columnAggregationLabel
  └─ Cell renderer:
    └─ IF aggregation row: calculateGrandTotal()
    └─ ELSE: calculateRowAggregation()
  └─ Add to columns array
↓
Return complete columns array
```

▮ 4. Table Data Building Flow

GridBuilder.ts Row Creation

```
Step 3: buildTableData(matrixData, props)
↓
Create regular rows from sortedRows:
  └─ Map each rowItem to GridRowData interface:
    └─ id: "row_{rowItem.id}"
    └─ rowItem: actual row entity
    └─ _index: array index
    └─ _isAggregation: undefined (regular row)
  └─ Store in tableData array
↓
IF enableRowAggregation:
  └─ Create aggregation row:
    └─ id: "aggregation_row"
```

- | — rowItem: null
- | — _index: based on aggregationPosition
- | — _isAggregation: true
- | — IF position === "top": unshift to beginning
- | — ELSE: push to end
- | — Update indices accordingly
- ↓
- Return complete tableData array

✂ 5. React Table Integration Flow

DynamicGrid.tsx Table Creation

```
buildGridData returns { tableData, columns }
↓
useReactTable({
  data: tableData,
  columns: columns,
  getCoreRowModel: getCoreRowModel()
})
↓
Table object created with:
| — Header groups and headers
| — Row model with data rows
| — Cell rendering contexts
| — Aggregation row flagged with _isAggregation
```

▯ 6. Rendering Flow

GridRenderer.tsx Component Tree

```
<GridRenderer table={table} />
↓
<table className="dynamic-grid">
| — <GridHeader table={table} />
|   | — Maps headerGroups
|   | — Maps headers in each group
|   | — Calls flexRender(header.column.columnDef.header)
|   | — Applies "header-cell" CSS classes
| — <GridBody table={table} />
|   | — Maps table.getRowModel().rows
|   | — Checks row.original._isAggregation
|   | — Applies "aggregation-footer" class if aggregation row
|   | — Maps row.getVisibleCells()
|   | — Calls flexRender(cell.column.columnDef.cell)
|   | — Applies aggregation CSS classes conditionally
| — </table>
```

7. Aggregation Calculation Flow

AggregationProcessor.ts Logic

```
WHEN aggregation cell renders:
↓
calculateColumnAggregation() OR calculateRowAggregation()
↓
Collect values from matrix:
├─ FOR column aggregation: iterate all rows in specific column
├─ FOR row aggregation: iterate all columns in specific row
├─ Get cellItem from matrix[rowKey][columnKey]
├─ Extract value using props.cellAttribute.get(cellItem)
├─ Handle Mendix value objects (displayValue/value)
├─ Filter out null/undefined values
↓
AggregationProcessor.calculate(values, functionName)
├─ Convert values to numbers where possible
├─ Select aggregation function (sum/avg/count/min/max/first/last)
├─ Execute calculation on numeric array
├─ Return calculated result
↓
AggregationProcessor.formatValue(result, formatString)
├─ Handle null/undefined results → "-"
├─ Apply format string if provided
├─ Default formatting for integers vs decimals
├─ Return formatted string
```

8. Content Rendering Flow

ContentRenderer.ts Logic

```
ContentRenderer.renderContent(item, showAs, attribute, textTemplate, widgets)
↓
SWITCH showAs:
├─ "attribute":
│   ├── Call attribute.get(item)
│   ├── Handle Mendix attribute objects
│   ├── Extract displayValue || value || String(value)
│   └─ Return string result
├─ "dynamicText":
│   ├── Call textTemplate.get(item)
│   ├── Extract value?.value || String(value)
│   └─ Return string result
├─ "custom":
│   ├── Call widgets.get(item)
│   └─ Return ReactElement
└─ DEFAULT: return ""
```

▮ 9. Update/Re-render Flow

Dependency Change Handling

```
User changes widget properties in Studio Pro
↓
Component re-renders with new props
↓
useMemo dependencies change detection:
├─ Data source changes → full rebuild
├─ Association changes → full rebuild
├─ Content type changes → column rebuild
├─ Aggregation setting changes → table data rebuild
└─ Label/format changes → aggregation cells re-render
↓
buildGridData() called again
↓
New table structure created
↓
React Table updates DOM efficiently
```

▮ 10. Error Handling Flow

Graceful Degradation

```
AT EACH STEP:
├─ DataProcessor: Try/catch for association access
├─ ContentRenderer: Try/catch for attribute access
├─ AggregationProcessor: Filter invalid values
├─ Value extraction: Handle various data types
└─ Fallback mechanisms for missing data

CONSOLE WARNINGS for:
├─ Failed association lookups
├─ Attribute access errors
├─ Unsupported aggregation functions
└─ Invalid cell values
```

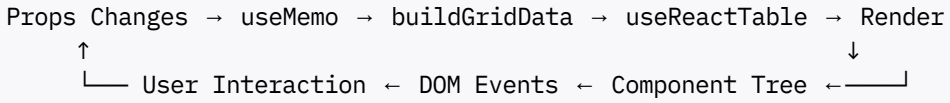
▮ 11. Key Data Flow Points

Critical Data Transformations

1. **Mendix Entities** → **Matrix Structure** (DataProcessor)
2. **Matrix + Props** → **Column Definitions** (ColumnBuilder)
3. **Sorted Arrays** → **Table Rows** (GridBuilder)
4. **Raw Values** → **Aggregated Results** (AggregationProcessor)
5. **Data + Config** → **Rendered Content** (ContentRenderer)

6. Table Data → DOM Elements (GridRenderer)

State Management Flow



This modular architecture ensures **clean separation of concerns**, **easy testing**, **maintainable code**, and **efficient re-rendering** while supporting complex CLEVR data relationships and aggregation functionality!